CISC 818 – Software Engineering with AI
Project Proposal: AI Study Companion

1. Project Title
AI Study Companion

2. Problem Statement
University students routinely struggle to manage large volumes of academic content across multiple concurrent courses. Research in educational psychology consistently shows that passive re-reading and manual note-taking — the dominant study strategies among undergraduates — are among the least effective methods for long-term retention. Students frequently devote more time to organizing materials than to genuine mastery of them, a pattern that directly undermines academic performance and exacerbates exam-related stress.

Existing tools such as NotebookLM offer general-purpose document interaction but are not purpose-built for the structured academic environment. They lack features critical to student success: adaptive quizzing, spaced repetition scheduling, course-level content organization, and collaborative study capabilities. The target users for this project are university students who regularly receive dense lecture materials and require a smarter, more efficient pathway from receiving content to mastering it. Addressing this gap is significant because improved study efficiency has measurable downstream effects on both academic outcomes and student well-being.

3. Proposed Solution
AI Study Companion is a web-based application that transforms lecture materials into interactive study experiences through large language model integration. The platform is structured around an academic hierarchy — Courses containing Lectures — mirroring how students naturally think about their workload. Upon uploading a PDF lecture file, the system automatically generates a structured summary, extracts key terms, and produces a flashcard set. Students then review those flashcards through a spaced repetition scheduler implementing the SM-2 algorithm, ensuring optimal review intervals based on individual performance. An adaptive multiple-choice quiz engine tracks weak areas by topic and adjusts question difficulty on subsequent attempts, providing targeted reinforcement where it is most needed.

Beyond individual study, the application supports a personal annotation system linked to each lecture and collaborative study groups where students can share flashcard sets using invite codes. As an exam date approaches, an AI-generated personalized study plan synthesizes identified weak areas and remaining time to produce a day-by-day review schedule. A progress dashboard with score trends and topic-level analytics provides students with an ongoing view of their readiness.

The intended user journey begins with registration and course creation. The student uploads lecture PDFs, triggers AI generation to receive an instant summary and flashcard set, reviews cards through spaced repetition, and takes a quiz. As the exam date nears, they receive a personalized study plan. Throughout, they annotate lectures with personal notes and collaborate with classmates in shared groups. The platform was built as a web application using React and TypeScript on the frontend, Node.js with Express on the backend, and PostgreSQL managed through Prisma ORM. The web platform was chosen deliberately: it requires no installation, is accessible from any device, and enables real-time collaboration features that would be significantly more complex to implement on a native mobile or desktop target.

4. Team Management

The four-member team is organized around architectural layers to balance ownership and enable parallel work. One developer owns the React frontend and UI components, while a second manages the Node.js/Express backend and Prisma database architecture. A third member specializes in AI integration, handling LLM prompt engineering, PDF processing, and adaptive quiz logic. The fourth leads Full-Stack/DevOps, overseeing Supabase, deployment, and shared tooling, while doubling as QA lead to resolve bottlenecks and handle cross-cutting tasks.

We coordinate via GitHub using a structured branching strategy: a protected main branch, a dev branch for integration, and role-prefixed feature branches (e.g., feat/FE-courses-page). All changes require peer-reviewed pull requests before merging to ensure collective ownership and code quality, with the QA/DevOps lead serving as a fallback reviewer. Weekly coordination includes a Monday sync for priorities, a mid-week Discord check-in, and Friday demos where completed work is reviewed before merging to dev. GitHub Issues tracks tasks, while standardized commit conventions (feat:, fix:, chore:) ensure a clean, reviewable history.

5. Timeline

The development schedule spans six weeks, with the final two weeks reserved for testing, polish, and presentation preparation.

Week 1 — Foundation & Setup. The first week establishes the development environment and deploys a working application skeleton. This includes initializing the React and Node.js projects, configuring Supabase for authentication and storage, defining the Prisma schema, and deploying both frontend and backend to Vercel and Railway respectively. The deliverable is a live URL where a user can register and log in.

Week 2 — Core AI Features. Week two implements the primary value proposition: PDF upload, AI-generated summaries, and flashcard generation. By the end of this week, a user can upload a lecture file and receive a structured summary with key topics and automatically generated flashcards stored in the database.

Week 3 — Quiz Engine & Spaced Repetition. The third week adds the adaptive quiz system and spaced repetition algorithm. The quiz engine generates multiple-choice questions, tracks incorrect answers by topic, and adjusts difficulty on subsequent attempts. The SM-2 algorithm schedules flashcard reviews, and a Due Today queue appears on the dashboard.

Week 4 — Study Plan, Notes & Collaboration. Week four completes remaining features: the AI-generated personalized study plan, the personal notes system linked to individual lectures, and collaborative study groups with invite codes. The progress dashboard with score trends and analytics is also completed, making the application feature-complete.

Week 5 — Testing & Refinement. The fifth week is dedicated to end-to-end testing, UI polish, and performance optimization. The team conducts a full manual QA pass, writes unit and integration tests for critical paths, and finalizes the AI usage log. The application is tagged as release v1.0.0.

Week 6 — Presentation Preparation & Submission. The final week is reserved for preparing the course presentation, rehearsing the live demo, and completing all submission requirements. The TA (@marcosmacedo) will be invited to the GitHub repository, the AI usage log will be finalized, and the team will conduct a full rehearsal of the presentation to ensure a polished, confident delivery on the presentation date.